

Task 3.1

Policy for attach(Observer *observer)

Ignores duplicate registrations as if the observer is already in the vector it won't be added again.

Policy for detach(Observer *observer)

If the observer is found in the vector it is removed, if not found it does nothing.

Task 3.2

The subject does not own its observers, and the observers don't detach themselves automatically. We make the client responsible for the lifetime of the observer.

Subjects' role: The subject class stores non-owning pointers to the observer objects; it can't delete them. Attach and detach are used for runtime registration and not for ownership status

Observers' role: An observer can't detach itself automatically this has to be done by the client

Overall, the client is the one responsible for observer being detached from subjects before being deleted. Allowing the client control ensures that no unexpected situations occur as the client can handle the attach and detach since cascading zone is complicated so it ensures that this process can be carefully controlled by the client

Task 3.5

We used a push implementation.

The Subject::notify(EventNotice ¬ice) method takes an eventnotice and passes it to the update method of each observer

Why we used the push model

Advantage: It is simple and can be easily tweaked, the subject pushes data to the observer with a message. The observer doesn't have to query and that's simplifies the observer implementation and makes interaction between the objects easier and more fluid.

Trade-off: Coupling. The subject and observer are coupled through the eventnotice object meaning If the format of the notice changes all observers will have to be updated. So, it is not as flexible.